



PATRONES

 SESIÓN 07 · UNIDAD II · ETHEREUM Y CONTRATOS

SOLIDITY II DATOS Y PATRONES

Los patrones de diseño de Solidity no son elegancia: son cicatrices de dinero que se perdió.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

Hoy el contrato custodia dinero ajeno. Todo gira alrededor de una pregunta: ¿qué pasa si alguien no juega limpio? Hay una pausa de 10 minutos entre A y B.

A**ESTRUCTURAS, ETHER Y PATRONES**

mapping, arreglos, herencia, enviar y recibir ether, retiro, CEI, pausa y gas medido.

~70 min**B****LABORATORIO 07 · SUBASTA SEGURA**

Completar la subasta, ver el ataque, desplegar y llenar la tabla de gas push/pull.

~75 min**C****DEVOLUCIÓN DEL ANTEPROYECTO**

Cómo se filtró el alcance, qué significa cada decisión y primeros pasos.

~25 min



AL SALIR DE AQUÍ

Manejar estructuras compuestas y aplicar patrones seguros de manejo de fondos, midiendo su costo en gas.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

- 01** ¿Por qué no se puede recorrer un mapping, y qué se hace en su lugar?
- 02** ¿Cómo se envía ether sin abrirle la puerta a un atacante?
- 03** ¿Por qué un contrato no debería enviar dinero por iniciativa propia?
- 04** ¿Cuánto cuesta la seguridad, medido en gas, y quién la paga?

RA3 · patrones y estándares (se evalúa) · RA5 · base de la seguridad (complementario, se profundiza en la S9)



SIGLAS Y TÉRMINOS DE HOY

MAPPING*tabla clave → valor*

Toda clave existe y devuelve cero si nunca se escribió. No se puede recorrer ni contar.

INTERFACE · LIBRARY*piezas para componer*

interface: solo firmas, para llamar contratos ajenos. library: funciones reutilizables sin estado.

CEI*Checks-Effects-Interactions*

Orden obligatorio: validar, cambiar el estado propio y, solo al final, llamar hacia fuera.

CIRCUIT BREAKER*interruptor · Pausable*

Detener funciones en una emergencia. Lo que nunca se detiene: retirar lo propio.

STRUCT · ENUM*tipos compuestos*

struct agrupa campos con nombre; enum define un conjunto cerrado de estados posibles.

RECEIVE · FALLBACK*funciones especiales*

Lo que ejecuta un contrato al recibir ether sin datos, o una llamada que no reconoce.

PULL OVER PUSH*patrón de retiro*

El contrato acredita saldos y cada quien retira lo suyo, en lugar de enviar pagos.

BPS*basis points · puntos básicos*

10 000 bps = 100 %. La forma de expresar porcentajes sin punto flotante.

A



APROXIMADAMENTE 70 MINUTOS

ESTRUCTURAS, ETHER Y PATRONES

La primera mitad es sobre datos. La segunda, sobre lo que pasa cuando esos datos son dinero y del otro lado hay un adversario.



UNA TABLA DONDE TODAS LAS CLAVES EXISTEN

```
mapping(address => uint256) public saldos;  
mapping(address => mapping(address => uint256)) public permisos; // anidado  
  
saldos[ana] = 10;  
saldos[beto];           // 0: nunca se escribió, pero "existe"
```

PROPIEDAD	CONSECUENCIA PRÁCTICA
Toda clave devuelve un valor	No se distingue «nunca escrito» de «escrito con cero». Por eso el certificado de la S6 llevaba el campo existe.
Las claves no se guardan	El valor vive en la ranura keccak256(clave, posición). No hay lista de claves que recorrer.
No tiene longitud	No se sabe cuántas entradas tiene ni cuáles son, ni desde dentro ni desde fuera.
Borrar = poner en cero	delete saldos[ana] no elimina la entrada: la devuelve al valor por defecto.

Error típico: usar `saldos[x] == 0` para decidir «esta cuenta no existe». Una cuenta que retiró todo también da cero.



MAPPING MÁS ARREGLO, Y EL BUCLE QUE NUNCA TERMINA

```
mapping(address => bool) public participo;  
address[] private _participantes;  
  
if (!participo[msg.sender]) {  
    participo[msg.sender] = true;           // ¿está? en un paso  
    _participantes.push(msg.sender);       // para poder listar  
}
```

El patrón habitual: el mapping responde «¿está?» en un paso y el arreglo permite listar. Es exactamente lo que hace Subasta.sol. Pero el arreglo esconde una trampa.

```
for (i = 0; i < _participantes.length; i++) { ... }
```

Cada vuelta cuesta gas. Con suficientes participantes, el bucle no cabe en el límite de gas del bloque y la función queda inutilizable para siempre.



REGLA QUE NO ADMITE EXCEPCIÓN

Ninguna función que se ejecute en una transacción recorre un arreglo que un usuario pueda hacer crecer. Leer la lista desde fuera, con una llamada view, es gratis y seguro. Recorrerla dentro de una transacción es una denegación de servicio esperando a ocurrir.

BORRAR DE UN ARREGLO SIN DEJAR HUECOS

```
uint256[] public ids;           // dinámico: crece con push
uint256[3] public podio;        // fijo: siempre 3

ids.push(7);
ids.pop();                      // quita el último

// delete NO lo quita: lo pone en 0 y deja un hueco
delete ids[1];

// quitar el elemento i sin hueco, en un paso (el orden cambia):
ids[i] = ids[ids.length - 1];
ids.pop();
```

- Ejemplo: `ids = [4, 9, 6]`. `delete ids[1]` deja `[4, 0, 6]` y la longitud sigue en 3.
- Con «intercambiar y quitar» sobre `i = 0`: `[6, 9]`. Cuesta lo mismo con 3 o con 3 000 elementos.
- Si el orden importa, casi siempre es mejor no borrar y marcar como inactivo.
- Arreglo fijo: más barato y más seguro cuando el tamaño se conoce de antemano.

En la literatura aparece como «swap and pop». Error típico: creer que `delete` acorta el arreglo.



STRUCT Y ENUM: UNA MÁQUINA DE ESTADOS

```
enum Estado { Abierta, PorFinalizar, Finalizada } // 0, 1, 2

struct Puja {
    address postor; // 20 bytes
    uint96 monto; // 12 bytes → juntos: una sola ranura
    uint64 momento; // otra ranura
}

function estado() public view returns (Estado) {
    if (finalizada) return Estado.Finalizada;
    if (block.timestamp >= fin) return Estado.PorFinalizar;
    return Estado.Abierta;
}
```

- El enum se guarda como un entero pequeño: por eso las pruebas comparan estado() con 0n, 1n y 2n.
- Prohíbe valores inválidos: no existe el estado 7.
- Aquí el estado se CALCULA desde el tiempo: nadie tiene que acordarse de cambiarlo.
- Un struct no tiene métodos: solo agrupa datos. El orden de los campos decide cuántas ranuras ocupa (A.16).

Error típico: guardar el estado en una variable y olvidar actualizarla cuando vence el plazo.

STORAGE, MEMORY Y CALldata

UBICACIÓN	QUÉ ES	CUÁNDO SE USA
storage	La ranura real y permanente del contrato. Escribir es lo más caro de la EVM.	Leer o modificar lo guardado: <code>struct storage c = _certificados[h];</code>
memory	Copia temporal, se borra al terminar la función.	Trabajar con datos que no se guardan.
calldata	Los datos de la transacción, solo lectura, sin copia.	Parámetros string o arreglos de funciones external: lo más barato.

BIEN · STORAGE

Certificado `storage c = _certificados[h];`
`c.revocado = true;`
 Apunta a la ranura real: queda guardado.

MAL · MEMORY

Certificado `memory c = _certificados[h];`
`c.revocado = true;`
 Es una copia: se pierde al salir. Compila sin avisar.

Por qué es tan peligroso: no hay error ni advertencia. La transacción termina bien, emite su evento y cobra su gas. Solo una prueba que vuelva a leer el estado descubre que no cambió nada: justo lo que la Sesión 8 enseña a escribir.

HERENCIA: HEREDAR LO AUDITADO

```
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";

contract Subasta is ISubasta, Ownable, Pausable {
    constructor(uint256 duracion, uint256 minima) Ownable(msg.sender) {
        ...
    }
    function pujar() external payable override whenNotPaused { ... }
}
```

PALABRA	SIGNIFICA
is A, B, C	Hereda estado, funciones y modificadores. Varios padres, del más general al más específico.
virtual · override	El padre permite que una función se reemplace; el hijo declara que la reemplaza.
Ownable(msg.sender)	Los argumentos del constructor del padre se pasan en la declaración del constructor hijo.

Error típico: copiar Ownable a mano «para entenderlo mejor». Se entiende leyéndolo; se usa heredándolo.



ABSTRACT, INTERFACE Y LIBRARY

	ABSTRACT CONTRACT	INTERFACE	LIBRARY
Qué es	Un contrato con funciones sin implementar.	Solo firmas de funciones, eventos y tipos.	Funciones reutilizables.
Tiene estado	Sí.	No.	No (salvo constantes).
Se despliega sola	No: hay que heredarla.	No.	Solo si tiene funciones external.
Para qué sirve	Plantillas con partes obligatorias.	Llamar a un contrato ajeno sabiendo solo su forma.	Aritmética, validaciones, utilidades.
En el laboratorio	—	ISubasta: la cumplen push, pull y Subasta.	Porcentajes, con using for.

[] POR QUÉ ISUBASTA IMPORTA EN EL LABORATORIO

PostorHostil recibe una ISubasta, no una Subasta concreta. Por eso el mismo atacante sirve contra la versión push y contra la pull: solo conoce la forma, y eso basta para compararlas.



PORCENTAJES SIN PUNTO FLOTANTE

```
library Porcentajes {
    uint256 internal constant BASE = 10_000; // 100 %
    function aplicarBps(uint256 monto, uint256 bps) internal pure returns (uint256) {
        return (monto * bps) / BASE; // multiplicar ANTES de dividir
    }
}
using Porcentajes for uint256;
mejorPuja.aplicarBps(100); // en vez de Porcentajes.aplicarBps(mejorPuja, 100)
```

CÁLCULO	RESULTADO	QUÉ ENSEÑA
1 % de 2 ETH = $2 \cdot 10^{18} \times 100 / 10\ 000$	$2 \cdot 10^{16}$ wei = 0,02 ETH	El mínimo siguiente tras una puja de 2 ETH es 2,02 ETH (lo comprueba una prueba).
$(150 \times 100) / 10\ 000$	1 wei	Correcto: se redondea hacia abajo.
$150 \times (100 / 10\ 000)$	0 wei	Dividir primero: $100 / 10\ 000 = 0$ en enteros. Todo se pierde.



TRANSFER, SEND Y CALL: CUÁL USAR

FORMA	GAS QUE ENTREGA AL RECEPTOR	SI FALLA	HOY
<code>transfer</code>	2 300 fijo	Revierte.	Desaconsejada: un receptor que haga algo al recibir puede quedarse sin gas.
<code>send</code>	2 300 fijo	Devuelve false, y es fácil ignorarlo.	Desaconsejada por la misma razón, y más peligrosa.
<code>call{value: x}("")</code>	Todo el disponible	Devuelve false.	La recomendada, SIEMPRE comprobando el resultado y aplicando CEI.

```
(bool ok, ) = payable(destino).call{value: monto}("");  
if (!ok) revert EnvioFallido();           // nunca ignorar ok
```

ERROR TÍPICO

Escribir `destino.call{value: monto}("");` sin leer el resultado. Si el envío falla, el contrato sigue como si hubiera pagado.

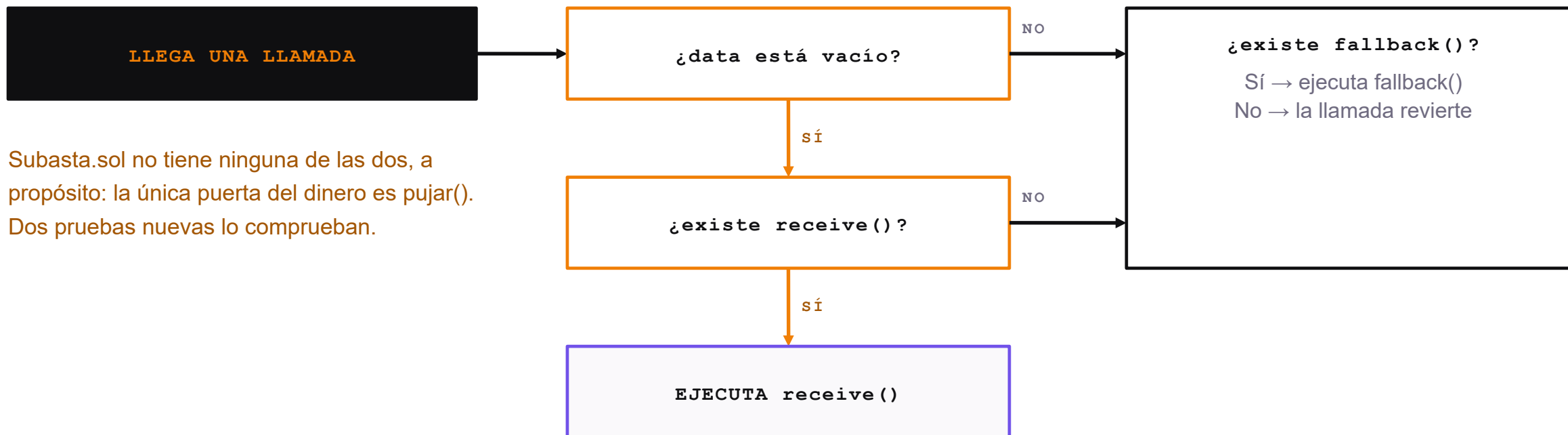


LOS 2 300 DE GAS DEJARON DE ALCANZAR

- | | | |
|-----------|----------------------------|---|
| 01 | ANTES DE 2019 | transfer se recomendaba contra la reentrada: con 2 300 de gas el receptor no alcanza a volver a llamar. |
| 02 | ISTANBUL · EIP-1884 | Una actualización de la red subió el costo de leer storage (SLOAD) de 200 a 800 de gas. |
| 03 | LO QUE SE ROMPIÓ | Contratos receptores legítimos cuyo receive leía una variable pasaron de caber a no caber en 2 300: transfer hacia ellos empezó a revertir. |
| 04 | LA LECCIÓN | No se puede depender de un costo de gas fijo: la red lo cambia. Se entrega todo el gas con call y la reentrada se previene con CEI. |

Fuentes: EIP-1884 (eips.ethereum.org/EIPS/eip-1884) y el artículo de ConsenSys Diligence «Stop Using Solidity's transfer() Now» (septiembre de 2019).

RECEIVE Y FALLBACK



Subasta.sol no tiene ninguna de las dos, a propósito: la única puerta del dinero es pugar(). Dos pruebas nuevas lo comprueban.

```

receive() external payable { ... }           // ether sin datos
fallback() external payable { ... }          // función que no existe
  
```



OWNABLE, Y LO QUE PASA SI SE PIERDE LA CLAVE

Un solo dueño con permisos especiales. En la subasta, el dueño es el beneficiario y quien puede pausar. Nada más: finalizar lo puede hacer cualquiera.

PIEZA DE OPENZEPELIN V5	QUÉ APORTA
<code>Ownable(msg.sender)</code>	El dueño se fija explícitamente en el constructor.
<code>onlyOwner</code>	Modificador que revierte con <code>OwnableUnauthorizedAccount(cuenta)</code> .
<code>transferOwnership</code>	Traspasa el control. Un error de tipeo en la dirección lo pierde para siempre.
<code>Ownable2Step</code>	El nuevo dueño debe llamar <code>acceptOwnership</code> : evita traspasar a una dirección equivocada.
<code>renounceOwnership</code>	Deja el contrato sin dueño. Irreversible: las funciones <code>onlyOwner</code> quedan bloqueadas.



DISEÑO MÍNIMO DE PRIVILEGIOS

¿Por qué `finalizar()` NO es `onlyOwner`? Si el dueño desaparece, los perdedores igual pueden cerrar la subasta y retirar.



EL CONTRATO NUNCA ENVÍA: CADA QUIEN RETIRA

PUSH · ENVIAR

Al superar una puja, el contrato devuelve el dinero al postor anterior en la misma transacción. Parece amable. Pero la puja nueva depende de que el pago al anterior salga bien, y ese anterior puede ser un contrato que lo rechace.

PULL · RETIRAR

El contrato anota: «a Ana se le deben 2 ETH». Ana llama a retirar() cuando quiera. Si su retiro falla, solo falla el suyo: nadie más queda bloqueado.



OCURRIÓ · KING OF THE ETHER THRONE, FEBRERO DE 2016

Un juego pagaba automáticamente al «rey» destronado con send y 2 300 de gas. Los pagos a billeteras que eran contratos fallaban por falta de gas, el contrato no revisaba el resultado y seguía como si hubiera pagado. Los autores devolvieron a mano 98,5 ether y publicaron un análisis del fallo (kingoftheether.com/postmortem.html) que se volvió referencia del patrón de retiro.



EL POSTOR HOSTIL

```
contract PostorHostil {  
    function pujarEn(ISubasta s) external payable {  
        s.pujar{value: msg.value}();  
    }  
    receive() external payable {  
        revert NoAceptoPagos(); // rechaza la devolución  
    }  
}
```

- | | | |
|-----------|-----------------------|---|
| 01 | PUJA | El contrato hostil puja 1 ETH en la subasta push. |
| 02 | BETO OFRECE 50 | La subasta intenta devolver 1 ETH al hostil dentro de la puja de Beto. |
| 03 | RECHAZO | receive() revierte, el call devuelve false, la subasta revierte con EnvioFallido. |
| 04 | CONGELADA | Nadie puede superar al hostil. Gana la subasta con 1 ETH. |

El atacante no robó un centavo. Solo se negó a que le devolvieran su dinero, y eso bastó para quedarse con el lote.



CHECKS → EFFECTS → INTERACTIONS

```
function retirar() external {
    uint256 monto = pendientes[msg.sender];
    if (monto == 0) revert NadaQueRetirar(); // 1 · CHECKS

    pendientes[msg.sender] = 0; // 2 · EFFECTS

    (bool ok, ) = payable(msg.sender).call{value: monto}(""); // 3 · INTERACTIONS
    if (!ok) revert EnvioFallido();
}
```

SI SE INVIERTE EL ORDEN

Enviar primero y poner a cero después. El receptor, dentro de su receive(), vuelve a llamar a retirar() antes de que el saldo quede en cero, y cobra otra vez. Y otra.

CON ESTE ORDEN

Cuando el receptor recibe el control, su saldo ya es cero. Si vuelve a llamar, no hay nada que cobrar. El ataque se desactiva sin ninguna herramienta extra.



PAUSAR, Y QUÉ NO SE PAUSA NUNCA

Pausable permite detener funciones cuando se descubre un problema. Es útil, y es un poder que se puede abusar.

QUÉ CONVIENE PAUSAR

- Recibir dinero nuevo: pujas, depósitos, compras.
- Funciones que agravan el daño si hay un fallo.
- Todo lo que el dueño no puede reparar sin detener.

QUÉ NO SE PAUSA

- Retirar lo propio. Si el dueño pudiera bloquear los retiros, el contrato sería una custodia disfrazada.
- Consultas: nunca dependen de la pausa.



LA PRUEBA QUE LO DEMUESTRA

En el laboratorio hay una prueba que pausa la subasta, comprueba que nadie puede pujar y comprueba que Ana Sí puede retirar. Si alguien pone `whenNotPaused` en `retirar()`, esa prueba falla con `EnforcedPause`.



EL ORDEN DE LAS VARIABLES CUESTA DINERO

DESORDENADO

```
// 3 ranuras
struct Desordenado {
    uint64 a;    // ranura 0
    uint256 b;   // ranura 1: no cabe con a
    uint64 c;    // ranura 2
}
```

ORDENADO

```
// 2 ranuras
struct Ordenado {
    uint64 a;    // ranura 0
    uint64 c;    // ranura 0: caben juntos
    uint256 b;   // ranura 1
}
```

88 341

gas para guardar el struct desordenado

66 279

gas para guardar los mismos datos,
ordenados

-22 062

una ranura nueva menos: 20 000 + 2 100 de
acceso en frío

Medido con `--gas-stats` (Empaquetado.sol). Regla: agrupar juntos los tipos pequeños. Error típico: creer que `uint64` ahorra por sí solo; solo ahorra si comparte ranura.



CONSTANT, IMMUTABLE Y CALldata

PALABRA	DÓNDE VIVE EL VALOR	EN SUBASTA.SOL	ERROR TÍPICO
constant	Incrustado en el código al compilar. No ocupa storage.	INCREMENTO_MINIMO_BPS = 100	Querer asignarla en el constructor: no compila.
immutable	Se fija en el constructor y queda incrustado en el código desplegado.	fin y pujaMinima	Usarla para algo que debe cambiar después.
calldata	Se lee directo de la transacción, sin copiarlo a memory.	(en la S6: string calldata programa)	Usar memory en parámetros de funciones external por costumbre.

POR QUÉ IMPORTA

Leer una variable de storage cuesta 2 100 de gas la primera vez en cada transacción (EIP-2929). Leer una constant o una immutable cuesta lo mismo que leer un número escrito en el código. Todo lo que no cambia después del despliegue debería ser una de las dos.



PUSH FRENTE A PULL, CON --GAS-STATS

Escenario A: cuatro postores distintos pujan 1, 2, 3 y 4 ETH; la subasta vence, se finaliza y todos retiran. Mismos pasos contra tres contratos.

GAS POR FUNCIÓN	SUBASTAPUSH	SUBASTAPULLMINIMA	SUBASTA (COMPLETA)
pujar · la primera	67 047	67 061	138 289
pujar · cada una de las otras 3	42 280	55 350	109 890
finalizar	56 227	71 622	73 922
retirar · 4 veces	—	29 977 c/u	30 000 c/u
TOTAL del escenario	250 114	424 641	661 881

El patrón seguro cuesta un 70 % más en total. Se paga con gusto: la alternativa es que un desconocido decida quién gana.



CUANDO DOS SE DISPUTAN EL LOTE

Escenario B: Ana y Beto se alternan seis pujas (1, 2, 3, 4, 5, 6 ETH). Ahora cada postor es superado varias veces.

PUJAR (SEGÚN --GAS-STATS)	SUBASTAPUSH	SUBASTAPULLMINIMA
mínimo	42 280	38 250
máximo (la primera)	67 047	67 061
TOTAL (con retiros)	334 674	454 064

Desde la cuarta puja, cada puja pull es MÁS BARATA que la push. ¿Por qué?

Estrenar pendientes[ana] cuesta 20 000; sumarle a una ranura que ya tiene valor, unos 2 900. Push, en cambio, paga el envío en cada puja.



LA LECTURA HONESTA

El total sigue siendo mayor en pull (un 36 % más), porque los retiros se pagan aparte. Pero la diferencia se achica cuanto más se disputa la subasta. El costo de un patrón no es un número fijo: depende de cómo se usa el contrato. Por eso se mide con el escenario real, no se supone.



SEIS REGLAS QUE CUESTA DINERO IGNORAR

01 Nunca recorrer, dentro de una transacción, un arreglo que un usuario pueda hacer crecer.

02 storage para modificar lo guardado; memory para una copia que se descarta.

03 Enviar ether con call, comprobar el resultado y aplicar CEI.

04 El contrato acredita; cada quien retira lo suyo.

05 Pausar lo que agrava el daño; nunca bloquear los retiros ajenos.

06 La seguridad cuesta gas. Se mide con el escenario real, se explica y se paga.



ANTES DE LA PAUSA: CUATRO PREGUNTAS

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 La subasta tiene 1 000 participantes en su arreglo. ¿Qué función NO deberían escribir nunca, y por qué?

02 Un contrato envía el pago con transfer y deja de funcionar tras una actualización de la red. ¿Qué pasó?

03 En retirar(), ¿qué línea tiene que ir antes del call, y qué ataque evita?

04 ¿Por qué la subasta pull cuesta más gas en el escenario A y la diferencia se achica en el B?

B



APROXIMADAMENTE 75 MINUTOS · EN PAREJAS

LABORATORIO 07 - SUBASTA SEGURA

Completar una subasta con patrón de retiro, ver el ataque contra la versión ingenua, desplegarla en Sepolia y medir cuánto cuesta defenderse.



DÓNDE ESTÁ CADA PATRÓN

ARCHIVO	QUÉ ES	PATRÓN QUE ILUSTRA
<code>contracts/s07/Subasta.sol</code>	La subasta completa. Ustedes trabajan sobre el andamiaje: 6 TODO.	Ownable, Pausable, retiro, CEI, enum, immutable
<code>ISubasta.sol</code> · <code>Porcentajes.sol</code>	Interfaz común y biblioteca de puntos básicos.	interface, library, using for
<code>SubastaPush.sol</code>	La versión ingenua. No se modifica.	antipatrón push
<code>SubastaPullMinima.sol</code>	Igual a la push salvo el patrón.	comparación justa de gas
<code>PostorHostil.sol</code>	El atacante.	receive que revierte
<code>Empaquetado.sol</code>	Dos structs con los mismos datos.	empaquetado de storage
<code>test/s07/Subasta.test.js</code>	23 pruebas: la especificación.	—
<code>test/s07/GasPushPull.test.js</code>	Los escenarios A y B de la tabla de gas.	medir, no suponer



COPIAR EL ANDAMIAJE Y VER EL ROJO

```
cd laboratorios-evm
cp andamiaje/s07/Subasta.sol contracts/s07/Subasta.sol      # PowerShell: igual
npx hardhat test test/s07/Subasta.test.js

7 passing
16 failing           ← así se empieza. Es lo esperado.
```

01 POR QUÉ 7 PASAN YA

Prueban partes que no dependen de los TODO: el constructor, la pausa de OpenZeppelin, receive/fallback, la push y el empaquetado.

02 SI NO COMPILA

El andamiaje compila tal cual. Si falla, se copió mal: repetir el cp. Si el error es de OpenZeppelin, falta npm install (S5).

03 EN CMD DE WINDOWS

```
copy andamiaje\s07\Subasta.sol contracts\s07\Subasta.sol
```

COMPLETAR LOS SEIS TODO, EN ORDEN

TODO	QUÉ HACER	PRUEBAS QUE LO CONFIRMAN
1 · estado	Finalizada si finalizada; PorFinalizar si block.timestamp >= fin; si no, Abierta.	«recorre los tres estados»
2 · minimoSiguiente	Sin postor: pujaMinima. Con postor: mejorPuja + 1 %.	«exige superar la puja anterior...»
3 · pujar · checks	Cerrada → SubastaCerrada. Monto bajo → Pujalnsuficiente(minimo, msg.value).	grupo «pujar»
4 · pujar · effects	ACREDITAR (+) al superado, nunca enviarle. Actualizar, registrar, emitir.	«acumula...» y ★ pull
5 · retirar	Leer → si 0, revertir → PONER EN CERO → call → evento.	grupo «retirar» y la de pausa
6 · finalizar	Vencida y no finalizada → marcar → acreditar al owner() → evento.	grupo «finalizar y estado»

Después de cada TODO: correr la suite. El número de «failing» debe bajar. Si sube, el último cambio rompió algo.



23 EN VERDE, Y LEER LO QUE IMPRIMEN

```
npx hardhat test test/s07/Subasta.test.js
```

```
S07 · push frente a pull · el postor hostil
```

- ✓ ★ contra la subasta PUSH, un postor hostil congela todas las pujas siguientes
 - ✓ ★ contra la subasta PULL, el mismo ataque no bloquea a nadie
- segunda puja · push: 42280 · pull: 55350 · retiro posterior: 29977

```
S07 · empaquetado de storage
```

```
desordenado: 88341 gas · ordenado: 66279 gas · ahorro: 22062
```

```
23 passing
```



PUNTO DE CONTROL · LEVANTEN LA MANO

Captura de pantalla de «23 passing»: es la primera evidencia del laboratorio. Si les da 22 y falla una sola, casi siempre es la de pausa o la de «acumula»: miren la tabla de errores (B.9) antes de preguntar.



VER EL ATAQUE CON LOS PROPIOS OJOS

```
npx hardhat test test/s07/Subasta.test.js --grep "postor hostil"
```

01 ABRIR LA PRUEBA ★ PUSH

Leer las cinco líneas: el hostil puja 1 ETH, Beto intenta 5 y 50, las dos revierten con EnvioFallido, el hostil sigue ganando.

02 ABRIR LA PRUEBA ★ PULL

Mismo ataque. Beto puja 5, la puja entra, y el ETH del hostil queda en pendientes: solo se perjudica a sí mismo.

03 ROMPERLO A PROPÓSITO

Cambien el += de pendientes por un call al superado. Corran: la ★ PULL falla. Deshagan el cambio.

04 ANOTAR

Pregunta 1 y 2 del informe: la cadena de llamadas, y quién pierde en cada caso.



SUBASTA EN SEPOLIA: ARRANCAR EL RELOJ

- | | | |
|-----------|-----------------------|---|
| 01 | REMIX | Crear ISubasta.sol, Porcentajes.sol y Subasta.sol en la MISMA carpeta. Compilar con 0.8.34 y optimización 200. |
| 02 | DESPLEGAR | Injected Provider, red Sepolia, billetera del curso. duracionSegundos = 600 · pujaMinima_ = 1000000000000000 (0,001 ETH). |
| 03 | DOS PUJAS | Desde dos cuentas: 0,001 ETH y luego 0,002 ETH (Value en la unidad correcta). Guardar los dos hashes. |
| 04 | MIENTRAS VENCE | Diez minutos: se usan para la tabla de gas (B.7). Después: finalizar, y retirar desde la superada y desde la dueña. |



SOLO RED DE PRUEBA

Billetera del curso, Sepolia, montos mínimos. No firmen nada cuyo contenido no entiendan: revisen red, función y valor en la ventana de la billetera.



LA TABLA DE GAS PUSH FRENTE A PULL

```
npx hardhat test test/s07/GasPushPull.test.js --grep "escenario A" --gas-stats
npx hardhat test test/s07/GasPushPull.test.js --grep "escenario B" --gas-stats
```

COPIAR DE --GAS-STATS	PUSH · A	PULL MÍNIMA · A	PUSH · B	PULL MÍNIMA · B
pujar · Average × #calls	_____	_____	_____	_____
finalizar	_____	_____	_____	_____
retirar · Average × #calls	—	_____	—	_____
TOTAL (sumar)	_____	_____	_____	_____

CÓMO COMPROBAR SUS CUENTAS

Cada prueba imprime el TOTAL exacto sumando los recibos. Su suma a partir de la tabla puede diferir en 1 o 2 de gas (el promedio se redondea); si difiere en más, revisen que multiplicaron por #calls.



EL INFORME DE DECISIONES DE DISEÑO

No es un resumen del laboratorio: es la defensa de por qué el contrato es como es. Por cada decisión, cinco columnas:

DECISIÓN	ALTERNATIVA DESCARTADA	POR QUÉ	EVIDENCIA	COSTO
Retiro (pull)	Devolver al instante (push)	Un receptor hostil congela la push	Pruebas ★	+70 % gas en A
pendientes = 0 antes del call	Poner en cero después	Evita la reentrada	Prueba de retiro doble	Ninguno
retirar() sin whenNotPaused	Pausar todo	...	...	...

- Mínimo cinco decisiones: las tres de la tabla y dos más (immutable, finalizar sin dueño, sin receive, registro de participantes...).
- Más las seis preguntas de la guía, la tabla de gas llena y los hashes de Sepolia.
- Máximo dos páginas. Se escribe en pareja; cada decisión, con la prueba o la cifra que la respalda.



LO QUE SUELE SALIR MAL

SÍNTOMA	CAUSA
Falla la de retiro: «changed by 0 wei»	Falta el call: el pendiente se pone en cero pero nunca se envía el dinero.
Retiro revierte con NadaQueRetirar la primera vez	Pusieron el pendiente en cero ANTES de leer el monto: el monto leído es 0.
«acumula si es superado varias veces» falla	Usaron = en lugar de += al acreditar al superado.
«EnforcedPause» al retirar	Pusieron whenNotPaused en retirar(): exactamente lo que no se debe.
El dueño no puede retirar tras finalizar	No acreditaron mejorPuja al owner() en finalizar().
La subasta no acepta la primera puja	minimoSiguiente() aplica el 1 % aunque no haya postor.
La ★ PULL falla con EnvioFallido	Devolvieron el dinero con call dentro de pujar(): eso es la push.
Remix no encuentra ISubasta.sol	Los tres archivos deben estar en la misma carpeta.



LO QUE EL LABORATORIO DEBIÓ DEJAR CLARO

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 ¿Qué prueba falla si alguien pone `whenNotPaused` en `retirar()`, y qué principio defiende esa prueba?

02 Si el pendiente se pone en cero DESPUÉS del `call`, ¿las 23 pruebas lo detectan? ¿Qué haría falta para detectarlo?

03 En la tabla de gas, ¿quién paga la diferencia entre `push` y `pull`: el ganador, los perdedores o el dueño?

04 ¿Qué prueba del laboratorio demuestra que la subasta no tiene `receive()`?

C

 APROXIMADAMENTE 25 MINUTOS

DEVOLUCIÓN DEL ANTEPROYECTO

Cada equipo recibe su decisión: aprobado, aprobado con recorte, o devuelto. Primero, los criterios con que se filtró el alcance.



EL FILTRO, EN EL ORDEN EN QUE SE APLICÓ

1 · LAS TRES PREGUNTAS QUE SE HACEN PRIMERO

- | | | |
|-----------|-----------------------|--|
| 01 | SIN TECNOLOGÍA | ¿Se puede enunciar el problema sin nombrar la tecnología? |
| 02 | ALTERNATIVA | ¿Qué alternativa sin cadena se consideró, y por qué no basta? (árbol de la S1) |
| 03 | UNA PÁGINA | ¿Cabe el alcance —lo que entra y lo que no— en una sola página? |

Si alguna respuesta es no, el anteproyecto vuelve. Después: devoluciones automáticas.
Por último: la rúbrica.

3 · La rúbrica del anteproyecto: material/proyecto/01-anteproyecto.pdf, sección 04.



2 · DEVOLUCIÓN AUTOMÁTICA

Sin importar el resto:

- Datos personales en la cadena, o su hash si se puede adivinar (Ley 1581 de 2012).
- Más de tres páginas.
- No hay lista de lo que queda fuera del alcance.
- Dinero real en cualquier parte del diseño.



APROBADO, RECORTADO O DEVUELTO

DECISIÓN	RECORTE TÍPICO Y SU CRITERIO	QUÉ SIGUE
Aprobado	—	El alcance aprobado es el contrato del equipo consigo mismo. Se construye desde hoy.
Aprobado con recorte	Token + marketplace + votación → solo el flujo que justifica la cadena. Integración con sistemas de la universidad → se simula. App móvil → una página web.	Aceptar o argumentar por escrito en 48 horas. Pasado el plazo, rige el recorte.
Devuelto	El árbol de la S1 dice «base de datos», o hay una devolución automática.	Se reorienta a un problema vecino; se califica la segunda versión.

UN RECORTE NO ES UN CASTIGO

El criterio es siempre el mismo: se conserva el flujo completo donde vive la regla que justifica la cadena, y sale lo que no la necesita o depende de algo que no controlan. Tres historias bien probadas valen más que diez a medias.

DEL PAPEL AL PRIMER CONTRATO

□ TRES PREGUNTAS PARA DISEÑAR EL CONTRATO DEL PROYECTO

1. ¿Mi contrato recibe o envía dinero? Si envía, ¿por qué no usa el patrón de retiro?
2. ¿Hay algún arreglo que un usuario pueda hacer crecer y que yo recorra en una transacción?
3. ¿Qué funciones pausaría en una emergencia, y cuáles nunca?

01 **ESTA SEMANA** Leer la devolución, aceptar o argumentar el recorte, y escribir en una lámina lo que queda fuera.

02 **ANTES DE LA S8** Primer borrador del contrato principal en Remix: compila, aplica dos patrones de hoy.

03 **EN LA S8** El borrador se mueve a un repositorio Hardhat con pruebas. Ese será su repositorio de todo el semestre.



CADA EQUIPO, ANTES DE SALIR

Respondan en parejas, sin mirar las láminas. Si alguna no sale en un minuto, esa es la lámina que hay que volver a ver.

01 ¿Qué paso del filtro fue el que más pesó en su decisión, y qué habrían cambiado?

02 Escriban en una frase qué queda FUERA de su alcance.

03 ¿Qué dato de su proyecto NO puede ir en la cadena, y qué va en su lugar?

04 ¿Qué patrón de hoy va a usar su contrato, y por qué?



LA PREGUNTA DE LA SEMANA

El postor hostil no robó nada y aun así ganó. ¿Dónde más —en el curso o en el mundo— alguien bloquea a todos solo con negarse a cooperar?

PARA LA PRÓXIMA SESIÓN

- Evidencia del laboratorio 07: 23 en verde, tabla de gas llena, Subasta en Sepolia (dirección + hashes) e informe de decisiones.
- Trabajo autónomo: refactorizar CertificadosUSB de la S6 aplicando dos patrones de hoy (guía, sección 11).
- Equipos: aceptar o argumentar el recorte en 48 h; primer borrador del contrato del proyecto.
- Traer el portátil con Node.js 22 y el repositorio instalado: la Sesión 8 es toda en la terminal.



LO QUE SE LLEVAN DE HOY

Un contrato que envía dinero depende de que el otro quiera recibirlo.

Hoy vieron que la seguridad tiene forma de orden de operaciones y de patrones concretos, y que cuesta gas medible. La próxima sesión salen de Remix y aprenden a demostrar, con pruebas automáticas, que el contrato hace lo que dice.

SESIÓN 8 · HARDHAT, PRUEBAS AUTOMATIZADAS Y DESPLIEGUE PROFESIONAL